Assignment 3: Pattern Mining and Recommender Systems

Task #3: Integration code

- **Group:** 38
- **Name:** Abin Shoby
- **Student ID:** a1933920

In [1]:
```
# Import libraries
import pandas as pd
import numpy as np
from sklearn.neighbors import NearestNeighbors
from mlxtend.frequent_patterns import apriori, association_rules
import random
import warnings
from collections import defaultdict
from collections import Counter
from IPython.display import display
from tqdm import tqdm
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from statsmodels.tsa.stattools import adfuller
warnings.filterwarnings("ignore")
seed = 10
random.seed(seed)
np.random.seed(seed)

pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)
```

1. Load and Preprocess Data

- Utility function to load a CSV dataset and remove null rows in the dataset.
- The columns are cast to the appropriate datatype for further usage.
- A recency weight is calculated based on the date to give importance to the most recent transaction.

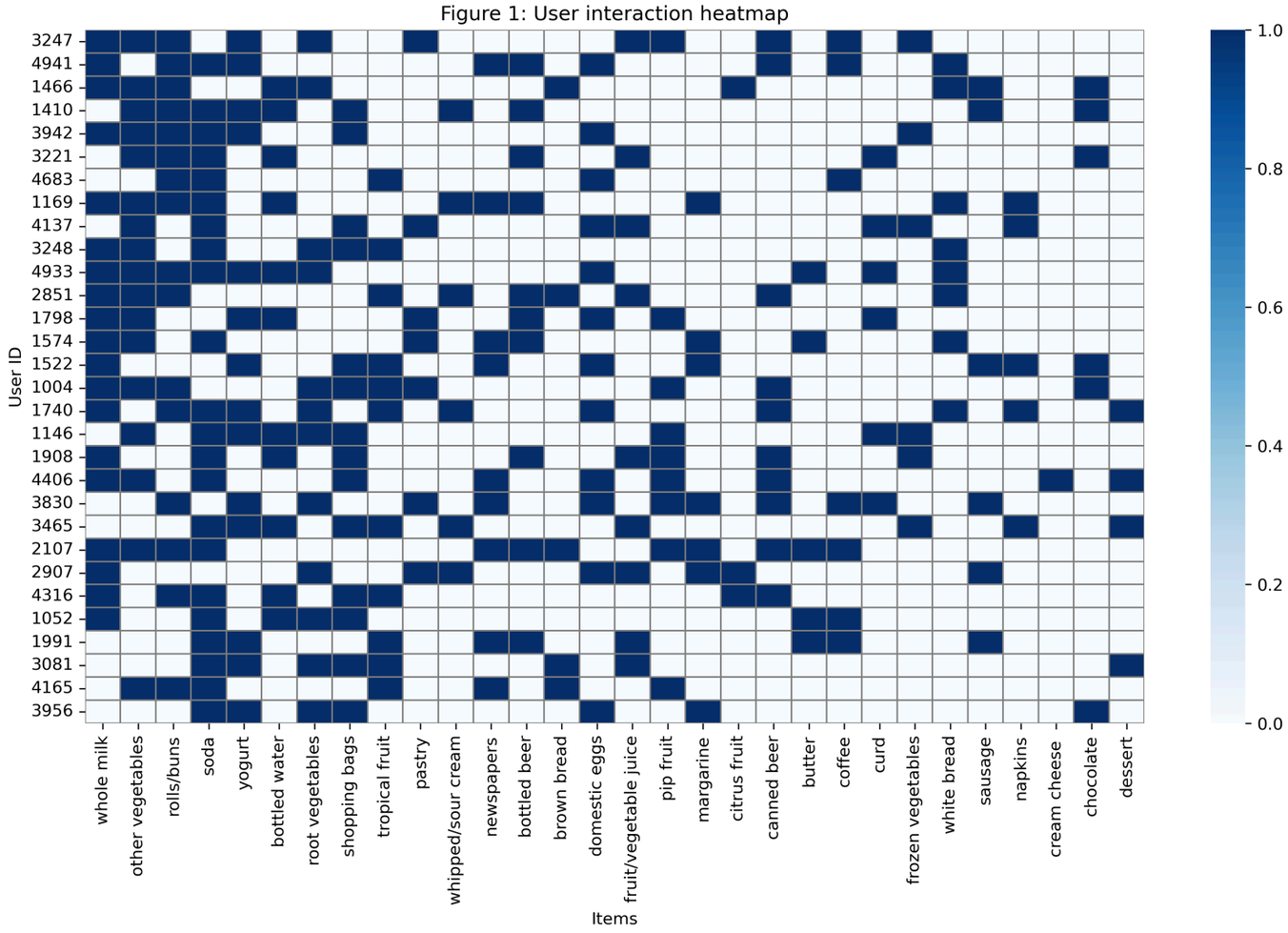
In [2]:
```
def load_data(file_path): # Task 1, Task 2, Task 3
    """
    Description:
        - Load train or test data.
        - Preprocess the dataset by resolving the datatypes.
    Params:
        - file_path: CSV data file path.
    Returns:
        - Preprocessed dataset
    """
    df = pd.read_csv(file_path, parse_dates=['Date'])
    # Remove Null rows.
    df = df.dropna()
    # Resolve datatypes.
    if 'User_id' in df.columns:
        # For train data.
        df['User_id'] = df['User_id'].astype(int)
    else:
        # For test data.
        df['User_id'] = df['user_id'].astype(int)
    df['year'] = df['year'].astype(int)
    df['month'] = df['month'].astype(int)
    df['day'] = df['day'].astype(int)
    df['day_of_week'] = df['day_of_week'].astype(int)
    df['Date'] = pd.to_datetime(df['Date'], format='%d/%m/%Y')
    # Indicates transaction recency
    df['recency'] = 1/(1+(pd.Timestamp.today() - df['Date']).dt.days)

    return df.sort_values(['User_id', 'Date'])
```

2. Exploratory Data Analysis

2.1 User-item Interaction Heatmap

In [3]:
```
# Heatmap of user–item interaction matrix
# Task 3
data = load_data('Groceries data train.csv')
interaction_matrix = data.drop_duplicates(subset=['User_id', 'itemDescription']) \
                        .assign(interacted=1) \
                        .pivot(index='User_id', columns='itemDescription', values='interacted') \
                        .fillna(0)

# sample top users and top items for clean heatmap
top_items = interaction_matrix.sum(axis=0).sort_values(ascending=False).head(30).index
top_users = interaction_matrix.sum(axis=1).sort_values(ascending=False).head(30).index
sample_matrix = interaction_matrix.loc[top_users, top_items]

# Heatmap
plt.figure(figsize=(12, 8), dpi=300)
sns.heatmap(sample_matrix, cmap='Blues', cbar=True, linewidths=0.5, linecolor='gray')
plt.title('Figure 1: User interaction heatmap')
plt.xlabel('Items')
plt.ylabel('User ID')
plt.tight_layout()
plt.show()
```

Figure 1: User interaction heatmap



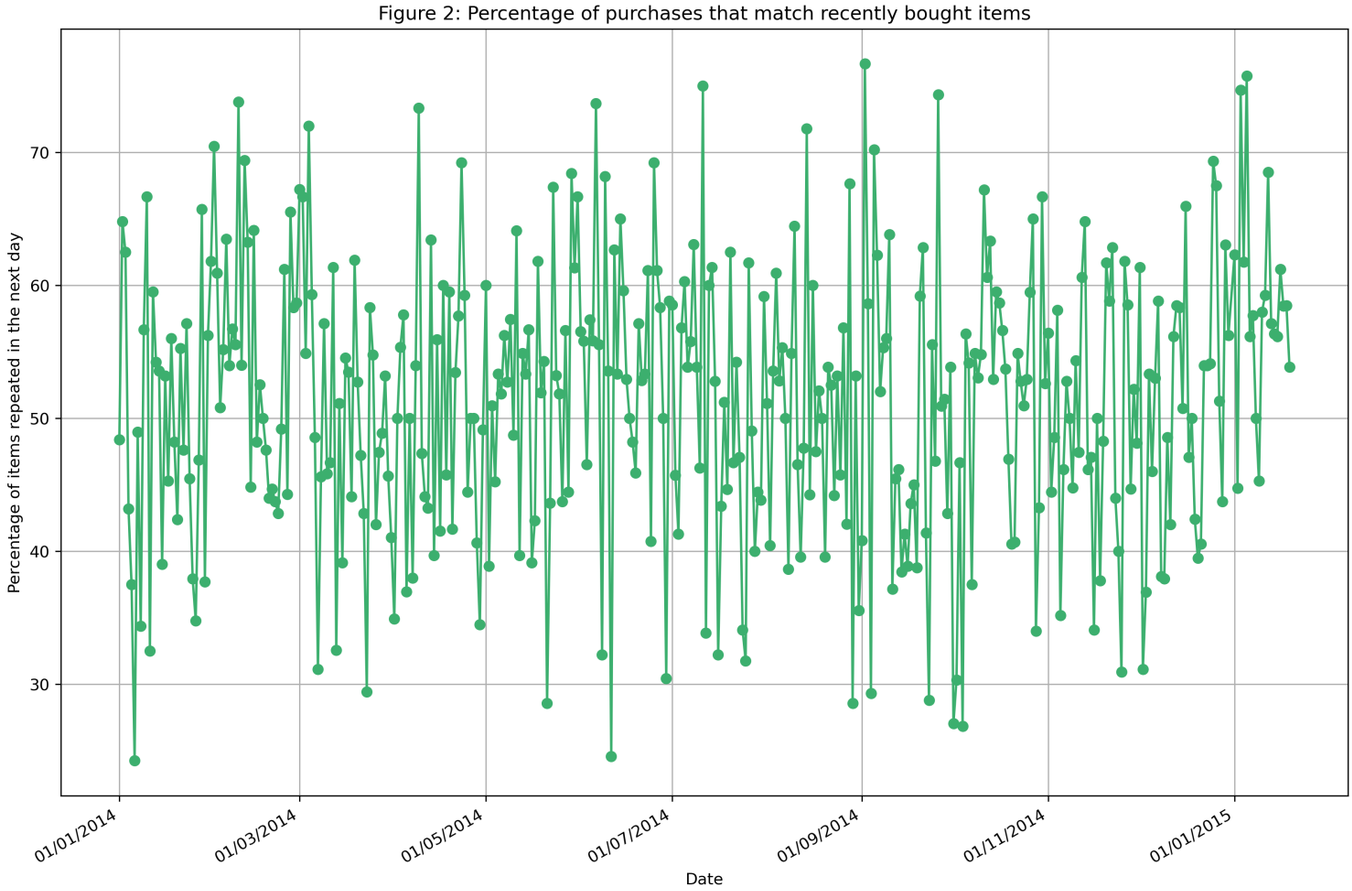
Observation

- The user-item interaction matrix is sparse, and some items are rarely purchased.
- Certain items, such as whole milk, other vegetables, and soda, are purchased by most users.

2.2 Recency Trend Analysis

In [4]:
```
# Task 3
data = data.sort_values(by='Date')
time_window = 1
recent_item_counts = []
dates = data['Date'].dt.date.unique()
for i in range(len(dates) - time_window):
    current_date = dates[i]
    time_end = dates[i + time_window]

    # Items bought currently
    today_items = set(data[data['Date'].dt.date == current_date]['itemDescription'])

    # Items bought on the next day
    next_days_data = data[(data['Date'].dt.date > current_date) & (data['Date'].dt.date <= time_end)]

    # Count items bought on the next day
    next_day_count = next_days_data['itemDescription'].isin(today_items).sum()
    total_count = len(next_days_data)

    if total_count > 0:
        recent_item_counts.append({
            'Date': current_date,
            'follow_up_hits': next_day_count,
            'total': total_count,
            'percentage': (next_day_count / total_count) * 100
        })

popularity_data = pd.DataFrame(recent_item_counts)
plt.figure(figsize=(12, 8), dpi=300)
plt.plot(popularity_data['Date'], popularity_data['percentage'], marker='o', color='mediumseagreen')
plt.title('Figure 2: Percentage of purchases that match recently bought items')
plt.xlabel('Date')
plt.ylabel('Percentage of items repeated in the next day')
plt.grid(True)
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%d/%m/%Y'))
plt.gcf().autofmt_xdate()
plt.tight_layout()
plt.show()
```

Figure 2: Percentage of purchases that match recently bought items



In [5]:
```
# ADF test for stationarity
# Task 3
trend_series = popularity_data['percentage'].dropna()
adf = adfuller(trend_series)
print(f"ADF Statistic: {adf[0]: .3f}")
print(f"p-value: {adf[1]: .3f}")
```

```
ADF Statistic: -18.183
p-value:  0.000
```

Observation

- The Augmented Dickey-Fuller (ADF) test can be used to check the stationarity of the above timeseries data (Dickey & Fuller 1979). If the p-value obtained from ADF is less than 0.05, then the timeseries is stationary.
- The p-value from the ADF test is 0, suggesting a stationary recency trend graph. It indicates that the repeated purchase trend follows a constant mean and variance. Therefore, the purchase of an item can be modelled linearly with purchase recency.

3. Split Dataset

The train data is split into a train and a dev set according to the Pareto Principle (Wikipedia 2025). Therefore, 80% of the training data is used for training, while the remaining samples are used as the dev set.

In [6]:
```
def split_data(file_path): # Task 1, Task 2, Task 3
    """
    Description:
        - Split the train dataset into to train and a dev set.
    Params:
        - file_path: CSV data file path.
    Returns:
        - Train and dev set.
    """
    df = load_data(file_path)
    train_list, dev_list = [], []
    for user_id, user_df in df.groupby('User_id'):
        split_idx = int(0.8 * len(user_df))
        train_user, dev_user = user_df.iloc[:split_idx], user_df.iloc[split_idx:]
        train_list.append(train_user)
        dev_list.append(dev_user)
    train_df = pd.concat(train_list)
    dev_df = pd.concat(dev_list)
    return train_df, dev_df
```

4. Modelling

This section contains the code for the following features:

- Frequent Pattern Mining (FPM) via Apriori algorithm.
- Collaborative Filtering (CF) via K-Nearest Neighbours (KNN) algorithm.
- Implementation of the hybrid recommender system using Frequent Pattern Mining and Collaborative Filtering for personalised recommendations (Lee, Kim & Rhee 2001).
- Utility functions for creating the user-item matrix, training and evaluating the models on the train and test sets, and hyperparameter tuning.
- Code for combining the pattern mining with collaborative filtering by filling gaps in the user-item matrix using the rules obtained from pattern mining (Saraei, Benali & Frayret 2025) (Parvatikar & Joshi 2015).
- Ranking the recommendations using a new scoring formula obtained by modifying the equation used to find the weighted rating by similar users (Krzywicki 2025, p. 12). In addition to this equation, they are weighted by recency to give significance to recently bought items.
- Computation of ranking metrics such as f1@k, precision@k and recall@k (Jadon & Patil 2025).
- Interface for providing the recommendations.

In [7]:
```
def frequent_pattern_mining(raw_input, min_support=0.01): # Task 1
    """
    Description:
        - Performs frequent pattern mining.
    Params:
```

```
        - raw_input: Transaction dataset.
        - min_support: Support threshold.
    Returns:
        - Scored frequent patterns represented by association rules.
    """
    # Encode transactions by user ID.
    basket = raw_input.groupby(['User_id', 'itemDescription']).size().unstack(fill_value=0)
    basket = basket.applymap(lambda x: 1 if x > 0 else 0)

    # Apply Apriori algorithm.
    frequent_itemsets = apriori(basket, min_support=min_support, use_colnames=True)
    if frequent_itemsets.empty:
        return pd.DataFrame()

    # Prevent random suggestions by using lift thresholding.
    rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.0)
    rules = rules.sort_values(by=['confidence', 'support'], ascending=[False, False])
    return rules
```

In [8]:
```
def build_user_item_matrix(df): # Task 2
    """
    Description:
        - Build a user-item matrix, a one-hot encoded user-item interaction matrix.
    Params:
        - df: Transaction dataframe.
    Returns:
        - user-item interaction matrix.
    """
    # Use support as rating values.
    n_transactions = df.groupby('User_id').size()
    user_item_counts = df.groupby(['User_id', 'itemDescription']).size().unstack(fill_value=0)
    user_item = user_item_counts.div(n_transactions, axis=0)

    return user_item
```

In [9]:
```
def train_knn_model(user_item_matrix, n_neighbors=5): # Task 2
    """
    Description:
        - Trains a KNN model.
    Params:
        - user_item_matrix: The user-item interaction matrix.
        - n_neighbors: The number of 'K' nearest neighbours required for the KNN model.
    Returns:
        - A trained KNN model.
    """
    knn = NearestNeighbors(metric='cosine', algorithm='brute', n_neighbors=n_neighbors)
    knn.fit(user_item_matrix)
    return knn
```

In [10]:
```
def collaborative_filter(raw_input, knn_model, user_item_matrix, transaction_data, top_k=5): # Task 2
    """
    Description:
        - Provide scored recommendations to the user using the Collaborative filtering algorithm.
    Params:
        - raw_input: The user's transaction history.
        - knn_model: Trained KNN model.
        - user_item_matrix: The user-item interaction matrix.
        - transaction_data: The transaction dataset.
        - top_k: No of recommendations required.
    Returns:
        - Scored recommendations.
    """
    # Apply KNN.
    distances, indices = knn_model.kneighbors([raw_input])
    distances = distances.flatten()
    numerators = defaultdict(float)
    denominators = defaultdict(float)

    neighbors = user_item_matrix.index[indices.flatten()]
    for i, neighbor in enumerate(neighbors):
        # Similarity from distance
        sim = 1 / (1 + abs(distances[i]))
        neighbor_data = transaction_data[transaction_data['User_id'] == neighbor]
        for _, row in neighbor_data.iterrows():
            item = row['itemDescription']
            # Not in purchase history.
            if raw_input.get(item, 0) > 0:
                continue

            # rating
            r_yi = user_item_matrix.at[neighbor, item] if item in user_item_matrix.columns else 0
            # Weighted by similarity and recency.
            recency = row['recency']
            numerators[item] += sim * r_yi * recency
            denominators[item] += sim

    # Scored based on recency, support and similarity.
    predicted_scores = {
        item: numerators[item] / denominators[item]
        for item in numerators if denominators[item] > 0
    }

    # Ranked items by score
    ranked = sorted(predicted_scores.items(), key=lambda x: x[1], reverse=True)
    return dict(ranked[:top_k])
```

In [11]:
```
def modify_user_item_matrix(user_item_matrix, rules): # Task 3
    """
    Description:
        - Modify the user-item interaction matrix by using the rules from pattern mining.
    Params:
        - user_item_matrix: The user-item interaction matrix.
        - rules: Association rules obtained through pattern mining.
    Returns:
        - A modified user-item matrix including association patterns.
    """

    user_item_matrix_mod = user_item_matrix.copy()

    for idx, rule in rules.iterrows():
        antecedents = list(rule['antecedents'])
        consequents = list(rule['consequents'])
        confidence = rule['confidence']

        # Matching priors.
        antecedent_mask = (user_item_matrix_mod[antecedents] > 0).all(axis=1)
        for consequent in consequents:
```

```
            # Modify the matrix.
            to_modify = antecedent_mask & (user_item_matrix_mod[consequent] == 0)
            # Fill missing items by confidence score.
            user_item_matrix_mod.loc[to_modify, consequent] = confidence

    return user_item_matrix_mod
```

In [12]:
```
def compute_topk_metrics(recommendations, ground_truth, k=5): # Task 3
    """
    Description:
        - Compute f1@k, recall@k and precision@k metrics.
    Params:
        - recommendations: Suggested recommendations.
        - ground_truth: Actual purchases.
        - k: Number of recommendations.
    Returns:
        - f1@k
        - recall@k
        - precision@k
    """
    if len(ground_truth) == 0:
        # No purchase history
        return 1, 1, 1
    if len(recommendations) == 0:
        # No recommendations
        return 0, 0, 0

    # No of true-positives.
    hits = [1 if item in ground_truth else 0 for item in recommendations[:k]]

    precision = sum(hits) / k
    recall = sum(hits) / len(ground_truth) if ground_truth else 0

    if precision + recall == 0:
        return 0, 0, 0
    f1 = 2 * precision * recall / (precision + recall)
    return f1, recall, precision
```

In [13]:
```
def evaluate(knn_model, user_item_matrix, transaction_df, eval_df, top_k=5, description="Evaluating with FPM"): # Task 3
    """
    Description:
        - Evaluate the recommendation system on a dataset.
    Params:
        - knn_model: The trained KNN model.
        - user_item_matrix: The user-item interaction matrix.
        - transaction_df: Transaction data.
        - eval_df: Evaluation dataset.
        - top_k: Number of recommendations.
        - description: Description for the progress bar.
    Returns:
        - Avg. f1@k
        - Avg. recall@k
        - Avg. precision@k
    """
    f1_scores = []
    recall_scores = []
    precision_scores = []
    avg_f1 = avg_precision = avg_recall = 0
    user_ids = eval_df['User_id'].unique()
    for _, user_id in tqdm(enumerate(user_ids), total=len(user_ids), desc=description, ncols=100):
        if user_id in user_item_matrix.index: # User has purchase history
            purchase_history = user_item_matrix.loc[user_id]
            purchased_item_names = purchase_history.index[purchase_history > 0]
            # Recommended items
            recs = collaborative_filter(purchase_history, knn_model, user_item_matrix, transaction_df, top_k)
            ground_truth = set(eval_df[eval_df['User_id'] == user_id]['itemDescription'].unique()) - set(purchased_item_names.tolist())
            # Compute metrics
            f1, recall, precision = compute_topk_metrics(list(recs.keys()), ground_truth, k=top_k)
            f1_scores.append(f1)
            recall_scores.append(recall)
            precision_scores.append(precision)
    if f1_scores:
        avg_f1 = np.mean(f1_scores)
    if recall_scores:
        avg_recall = np.mean(recall_scores)
    if precision_scores:
        avg_precision = np.mean(precision_scores)

    return avg_f1, avg_recall, avg_precision
```

In [14]:
```
def recommend_with_fpm_only(rules, raw_input, top_k=5): # Task 1
    """
    Description:
        - Evaluate the Frequent Pattern Mining recommendation.
    Params:
        - rules: Association rules obtained via FPM.
        - raw_input: The user purchase history.
        - top_k: Number of recommendations.
    Returns:
        - Scored recommendations based on confidence.
    """
    purchase_history = set(raw_input)
    # Sort by confidence and support
    rules = rules.sort_values(by=['confidence', 'support'], ascending=[False, False])
    predicted_scores = {}
    for _, row in rules.iterrows():
        antecedent = row['antecedents']
        consequent = row['consequents']
        # Matching rules.
        if antecedent.issubset(purchase_history):
            # Recommend unseen items.
            rcs = (set(consequent) - set(predicted_scores.keys())) - purchase_history
            for item in rcs:
                predicted_scores[item] = row['confidence']
        if len(predicted_scores) >= top_k:
            break
    # Top-k items
    ranked = sorted(predicted_scores.items(), key=lambda x: x[1], reverse=True)
    return dict(ranked[:top_k])
```

In [15]:
```
def evaluate_fpm(rules, train_df, eval_df, top_k=5, description="Evaluating with only FPM"): # Task 3
    """
    Description:
        - Evaluate Frequent Pattern Mining recommendations
    Params:
        - rules: Association rules.
        - train_df: Train dataset.
```

```
        - eval_df: Evaluation dataset.
        - top_k: Number of recommendations.
        - description: Description for the progress bar.
    Returns:
        - Avg. f1@k
        - Avg. recall@k
        - Avg. precision@k
    """
    test_transactions = eval_df.groupby('User_id')['itemDescription'].apply(list)
    train_transactions = train_df.groupby('User_id')['itemDescription'].apply(list)
    f1_scores = []
    recall_scores = []
    precision_scores = []
    avg_f1 = avg_precision = avg_recall = 0
    user_ids = eval_df['User_id'].unique()
    # Evaluate for each user
    for _, user_id in tqdm(enumerate(user_ids), total=len(user_ids), desc=description, ncols=100):
        if user_id in train_transactions.index:
            purchased_item_names = train_transactions.loc[user_id]
            ground_truth = set(test_transactions.loc[user_id]) - set(purchased_item_names)
            # Get recommendations
            recs = recommend_with_fpm_only(rules, purchased_item_names, top_k)
            # Metrics
            f1, recall, precision = compute_topk_metrics(list(recs.keys()), ground_truth, k=top_k)
            f1_scores.append(f1)
            recall_scores.append(recall)
            precision_scores.append(precision)
    if f1_scores:
        avg_f1 = np.mean(f1_scores)
    if recall_scores:
        avg_recall = np.mean(recall_scores)
    if precision_scores:
        avg_precision = np.mean(precision_scores)

    return avg_f1, avg_recall, avg_precision
```

In [16]:
```
def hyperparameter_tuning(train_df, dev_df, top_k=5): # Task 3
    """
    Description:
        - Perform hyperparameter tuning of the recommendation system on the dev set using the Grid Search algorithm.
    Params:
        - train_df: Train dataset.
        - dev_df: Dev dataset.
        - top_k: No of recommendations.
    Returns:
        - best_params_with_fpm: Best parameters for the system with frequent pattern mining.
        - best_params_without_fpm: Best parameters for the system without frequent pattern mining.
    """
    # Best f1 and parameters with frequent pattern mining.
    best_f1_with_fpm = -1
    best_params_with_fpm = None
    # Best f1 and parameters without frequent pattern mining.
    best_f1_without_fpm = -1
    best_params_without_fpm = None
    user_item_matrix = build_user_item_matrix(train_df)

    # Perform grid search.
    for k in [7, 5, 3]:
        # Train
        print("Training KNN model...")
        knn_model = train_knn_model(user_item_matrix, n_neighbors=k)

        print("Evaluating the models...")
        # Evaluate without FPM.
        avg_f1, _, _ = evaluate(knn_model, user_item_matrix, train_df, dev_df, top_k=top_k, description="Evaluating without FPM")
        if avg_f1 > best_f1_without_fpm:
            best_f1_without_fpm = avg_f1
            best_params_without_fpm = (k, 1)

        for i, min_support in enumerate([0.1, 0.01, 0.001]):
            print(f"Using K={k}, min_support={min_support}")
            rules = frequent_pattern_mining(train_df, min_support=min_support)
            user_item_matrix_fpm = modify_user_item_matrix(user_item_matrix, rules)

            # Evaluate with fpm
            avg_f1_fpm, _, _ = evaluate(knn_model, user_item_matrix_fpm, train_df, dev_df, top_k=top_k, description="Evaluating with FPM")

            print(f"Avg f1@{top_k} on dev set with FPM: {avg_f1_fpm*100:.2f}%")
            print(f"Avg f1@{top_k} on dev set without FPM: {avg_f1*100:.2f}%")

            if avg_f1_fpm > best_f1_with_fpm:
                best_f1_with_fpm = avg_f1_fpm
                best_params_with_fpm = (k, min_support)
    print(
        f"Best Parameters for CF with FPM:\n",
        "======================================\n",
        f"K={best_params_with_fpm[0]}\n",
        f"min_support={best_params_with_fpm[1]}\n",
        f"f1@{top_k}={best_f1_with_fpm*100:.2f}%"
    )
    print(
        f"Best Parameters for CF without FPM:\n",
        "======================================\n",
        f"K={best_params_without_fpm[0]}\n",
        f"f1@{top_k}={best_f1_without_fpm*100:.2f}%"
    )
    return best_params_with_fpm, best_params_without_fpm
```

In [17]:
```
def get_support_confidence(antecedent, consequent, df): # Task 1
    """
    Description:
        - Compute the support and confidence score for each pattern on a dataset.
    Params:
        - antecedent: The antecedent of a pattern.
        - consequent: The consequent of a pattern.
        - df: The evaluation dataset.
    Returns:
        - The support and confidence score of a pattern.
    """
    antecedent_count = 0
    rule_count = 0
    total_trans = len(df)

    for trans in df:
        if antecedent.issubset(set(trans)):
            antecedent_count += 1
            if consequent.issubset(set(trans)):
```

```
                rule_count += 1

    support = rule_count / total_trans
    confidence = rule_count / antecedent_count if antecedent_count > 0 else 0
    return support, confidence
```

In [18]:
```
def get_topk_patterns(rules, train_df, test_df, top_k=5): # Task 1
    """
    Description:
        - Get top-k patterns from the association rules.
        - Compute the support and confidence of these patterns on the train and test sets.
        - Sort the patterns by confidence score.
    Params:
        - rules: The association rules obtained via pattern mining.
        - train_df: The training dataset.
        - test_df: The test dataset.
        - top_k: To select top-k patterns.
    Returns:
        - Top-k patterns from rules sorted by confidence score.
    """
    # Test data transactions
    test_transactions = test_df.groupby('User_id')['itemDescription'].apply(list).tolist()
    patterns = []

    for _, row in rules.iterrows():
        antecedent = row['antecedents']
        consequent = row['consequents']
        # Compute support and confidence on the test set.
        sup_test, conf_test = get_support_confidence(antecedent, consequent, test_transactions)

        patterns.append({
            'antecedents': antecedent,
            'consequents': consequent,
            'support_train': row['support'],
            'confidence_train': row['confidence'],
            'support_test': sup_test,
            'confidence_test': conf_test
        })

    patterns_df = pd.DataFrame(patterns)
    # Sort by confidence score.
    top_patterns = patterns_df.sort_values(
        by=['confidence_train', 'support_train', 'confidence_test', 'support_test'],
        ascending=[False, False, False, False]
    ).head(top_k)
    return top_patterns
```

In [19]:
```
class RecommendationSystem: # Task 3
    """
    Description:
        A Recommendation System with the following features:
            - A basic text interface to provide a User ID and an option to choose between
            recommendations with pattern mining and without it.
            - Integrates Apriori (Pattern mining) and KNN (Collaborative filtering) algorithms.
            - Train KNN and mine association rules using Apriori.
            - Hyperparameter tuning.
            - Evaluates the test dataset using f1@k, precision@k and recall@k metrics.
            - Provides top 5 recommendations with scores.
    """
    def __init__(self, train_file, test_file, top_k = 5):
        """
        Description:
            Initialise the dataset and hyperparameters.
        Params:
            - train_file: Train dataset path.
            - test_file: Test dataset path.
            - top_k: No of recommendations.
        """
        self.top_k = top_k
        self.train_df, self.dev_df = split_data(train_file)
        self.test_df = load_data(test_file)
        self.build_complete = False
        self.with_fpm_k = self.without_fpm_k = 5
        self.with_fpm_min_supp = 0.001

    def build_model(self):
        """
        Description:
            Build the KNN model and generate rules using the Apriori algorithm.
            Also, performs hyperparameter tuning.
        """
        # Hyperparameter tuning
        print("Finding optimal parameters...")
        with_fpm_params, without_fpm_params = hyperparameter_tuning(self.train_df, self.dev_df, top_k=self.top_k)
        self.with_fpm_k, self.with_fpm_min_supp = with_fpm_params
        self.without_fpm_k, _ = without_fpm_params

        # Train on the full train dataset using the optimal parameters.
        print("Training on full data using optimal parameters...")
        self.full_train_df = pd.concat([self.train_df, self.dev_df])
        self.user_item_matrix = build_user_item_matrix(self.full_train_df)

        # Generate association rules.
        self.rules = frequent_pattern_mining(self.full_train_df, min_support=self.with_fpm_min_supp)
        # Create a modified user-item matrix using the rules.
        self.user_item_matrix_fpm = modify_user_item_matrix(self.user_item_matrix, self.rules)

        self.knn_model_without_fpm = train_knn_model(self.user_item_matrix, n_neighbors=self.without_fpm_k)
        self.knn_model_with_fpm = train_knn_model(self.user_item_matrix, n_neighbors=self.with_fpm_k)
        print("Models successfully built.")

        self.build_complete = True

    def get_recommendations(self, user_id, with_fpm):
        """
        Description:
            - Get recommendations for the given User ID.
        Params:
            user_id: Input User ID.
            with_fpm: With or without frequent pattern mining.
        Returns:
            - Scored recommendations.
        """

        if user_id not in self.test_df['User_id'].tolist():
            # Invalid user
            print(f"User {user_id} does not exist in the database. Please try another user id.")
```

```
            return None
        if user_id not in self.full_train_df['User_id'].tolist():
            # No purchase history
            print(f"User {user_id} does not have a purchase history.")
            return None

        purchase_history = self.user_item_matrix.loc[user_id]
        recommendations = None
        if with_fpm:
            # With frequent pattern mining.
            recommendations = collaborative_filter(
                purchase_history, self.knn_model_with_fpm, self.user_item_matrix_fpm, self.full_train_df, self.top_k
            )
        else:
            # Without frequent pattern mining.
            recommendations = collaborative_filter(
                purchase_history, self.knn_model_without_fpm, self.user_item_matrix, self.full_train_df, self.top_k
            )
        if not recommendations:
           print(f"No new items to recommend for the User {user_id} since the user does not have enough purchase history.")
        return recommendations

    def get_test_data_performance(self):
        """
        Description:
            - Obtain the test data performance of the system.
        """
        with_fpm_f1, with_fpm_recall, with_fpm_precision = evaluate(
            self.knn_model_with_fpm,
            self.user_item_matrix_fpm,
            self.full_train_df,
            self.test_df,
            top_k=self.top_k,
            description="Evaluating with FPM"
        )
        without_fpm_f1, without_fpm_recall, without_fpm_precision = evaluate(
            self.knn_model_without_fpm,
            self.user_item_matrix,
            self.full_train_df,
            self.test_df,
            top_k=self.top_k,
            description="Evaluating without FPM"
        )
        only_fpm_f1, only_fpm_recall, only_fpm_precision = evaluate_fpm(
            self.rules,
            self.full_train_df,
            self.test_df,
            top_k=self.top_k,
            description="Evaluating with only FPM"
        )
        print("Performance of Recommendation system on Test data with Frequent Pattern Mining")
        print("================================================================================")
        print(f"f1@{self.top_k}: {with_fpm_f1*100:.2f}%")
        print(f"recall@{self.top_k}: {with_fpm_recall*100:.2f}%")
        print(f"precision@{self.top_k}: {with_fpm_precision*100:.2f}%")
        print("Performance of Recommendation system on Test data without Frequent Pattern Mining")
        print("================================================================================")
        print(f"f1@{self.top_k}: {without_fpm_f1*100:.2f}%")
        print(f"recall@{self.top_k}: {without_fpm_recall*100:.2f}%")
        print(f"precision@{self.top_k}: {without_fpm_precision*100:.2f}%")
        print("Performance of Recommendation system on Test data only with Frequent Pattern Mining")
        print("================================================================================")
        print(f"f1@{self.top_k}: {only_fpm_f1*100:.2f}%")
        print(f"recall@{self.top_k}: {only_fpm_recall*100:.2f}%")
        print(f"precision@{self.top_k}: {only_fpm_precision*100:.2f}%")

    def get_pattern_recommendation_examples(self):
        """
        Description:
            - Display the top five patterns using Pattern Mining.
            - Display two example recommendations for each top frequent pattern.
        """
        # Top 5 frequent patterns on the train set
        top_patterns = get_topk_patterns(self.rules, self.full_train_df, self.test_df, top_k=5)
        recommendations = []
        user_groups = self.full_train_df.groupby('User_id')['itemDescription'].apply(set) # Train set transactions
        test_user_groups = self.test_df.groupby('User_id')['itemDescription'].apply(set) # Test set transactions
        # Get examples for each pattern
        for i, rule in top_patterns.iterrows():
            antecedents = rule['antecedents']
            consequents = rule['consequents']
            score = rule['confidence_train']
            matched_users = []

            # Get valid users with matching antecedents
            for user_id, items in test_user_groups.items():
                if antecedents.issubset(items):
                    matched_users.append(user_id)

            # Two sample recommendations
            recs = []
            for uid in matched_users[:2]:
                purchased = set()
                if uid in user_groups.index:
                    purchased = set(user_groups[uid]) # Already purchased items.
                recs.append({
                    'pattern': str(set(antecedents)) + ' => ' + str(set(consequents)),
                    'User_id': uid,
                    'recommended_items': list(set(consequents) - purchased),
                    'score': score
                })

            recommendations.extend(recs)
        recommendations = pd.DataFrame(recommendations)
        print("Top 5 patterns")
        print("================================================================================")
        display(top_patterns)

        print("Example recommendations using the top 5 patterns")
        print("================================================================================")
        display(recommendations)

    def run(self):
        """
        Description:
            - Provides a user interface.
        """
```

```
        if not self.build_complete:
            # Train and build the model.
            self.build_model()

        user_id = int(input("Enter your User ID: "))
        option = input(
            "Do you prefer recommendations with or without Frequent Pattern Mining (Apriori) (with/without)?"
        )
        recommendations = self.get_recommendations(user_id, option.strip().lower() in ['with', 'yes'])
        if recommendations:
            print(f"Top {self.top_k} recommendations for user {user_id} {option} Frequent Pattern Mining:")
            print("=============================================================================================")
            rec_df = []
            for item in recommendations.items():
                rec_df.append([item[0], item[1]])
            rec_df = pd.DataFrame(rec_df, columns=['Items', 'Score'])
            print(rec_df)
```

5. Train and Build Models

In [20]:
```
recommendation_sys = RecommendationSystem("Groceries data train.csv", "Groceries data test.csv", top_k=5)
recommendation_sys.build_model()
```

```
Finding optimal parameters...
Training KNN model...
Evaluating the models...
Evaluating without FPM: 100%|██████████| 3493/3493 [00:12<00:00, 288.78it/s]
Using K=7, min_support=0.1
Evaluating with FPM: 100%|██████████| 3493/3493 [00:11<00:00, 298.44it/s]
Avg f1@5 on dev set with FPM: 4.91%
Avg f1@5 on dev set without FPM: 4.91%
Using K=7, min_support=0.01
Evaluating with FPM: 100%|██████████| 3493/3493 [00:13<00:00, 263.23it/s]
Avg f1@5 on dev set with FPM: 47.40%
Avg f1@5 on dev set without FPM: 4.91%
Using K=7, min_support=0.001
Evaluating with FPM: 100%|██████████| 3493/3493 [00:15<00:00, 224.14it/s]
Avg f1@5 on dev set with FPM: 96.30%
Avg f1@5 on dev set without FPM: 4.91%
Training KNN model...
Evaluating the models...
Evaluating without FPM: 100%|██████████| 3493/3493 [00:10<00:00, 348.82it/s]
Using K=5, min_support=0.1
Evaluating with FPM: 100%|██████████| 3493/3493 [00:10<00:00, 345.53it/s]
Avg f1@5 on dev set with FPM: 4.50%
Avg f1@5 on dev set without FPM: 4.50%
Using K=5, min_support=0.01
Evaluating with FPM: 100%|██████████| 3493/3493 [00:10<00:00, 320.47it/s]
Avg f1@5 on dev set with FPM: 47.19%
Avg f1@5 on dev set without FPM: 4.50%
Using K=5, min_support=0.001
Evaluating with FPM: 100%|██████████| 3493/3493 [00:12<00:00, 283.44it/s]
Avg f1@5 on dev set with FPM: 96.30%
Avg f1@5 on dev set without FPM: 4.50%
Training KNN model...
Evaluating the models...
Evaluating without FPM: 100%|██████████| 3493/3493 [00:08<00:00, 422.74it/s]
Using K=3, min_support=0.1
Evaluating with FPM: 100%|██████████| 3493/3493 [00:08<00:00, 418.35it/s]
Avg f1@5 on dev set with FPM: 3.58%
Avg f1@5 on dev set without FPM: 3.58%
Using K=3, min_support=0.01
Evaluating with FPM: 100%|██████████| 3493/3493 [00:09<00:00, 383.91it/s]
Avg f1@5 on dev set with FPM: 47.00%
Avg f1@5 on dev set without FPM: 3.58%
Using K=3, min_support=0.001
Evaluating with FPM: 100%|██████████| 3493/3493 [00:10<00:00, 345.52it/s]
Avg f1@5 on dev set with FPM: 96.29%
Avg f1@5 on dev set without FPM: 3.58%
Best Parameters for CF with FPM:
 ====================================
 K=7
 min_support=0.001
 f1@5=96.30%
Best Parameters for CF without FPM:
 ====================================
 K=7
 f1@5=4.91%
Training on full data using optimal parameters...
Models successfully built.
```

6. Evaluate Model Performance

In [21]:
```
# Test data performance
recommendation_sys.get_test_data_performance()
```

```
Evaluating with FPM: 100%|██████████| 3231/3231 [00:15<00:00, 213.98it/s]
Evaluating without FPM: 100%|██████████| 3231/3231 [00:10<00:00, 300.34it/s]
Evaluating with only FPM: 100%|██████████| 3231/3231 [08:34<00:00,  6.28it/s]
Performance of Recommendation system on Test data with Frequent Pattern Mining
================================================================================
f1@5: 97.42%
recall@5: 97.42%
precision@5: 97.42%
Performance of Recommendation system on Test data without Frequent Pattern Mining
================================================================================
f1@5: 7.81%
recall@5: 8.57%
precision@5: 8.52%
Performance of Recommendation system on Test data only with Frequent Pattern Mining
================================================================================
f1@5: 18.68%
recall@5: 20.90%
precision@5: 20.03%
```

7. Recommendation Examples

Case 1: Recommendation example with Frequent Pattern Mining and Collaborative Filtering

In [22]:
```
# Valid user and with FPM.
recommendation_sys.run()
```

```
Top 5 recommendations for user 1000 with Frequent Pattern Mining:
================================================================================
           Items     Score
0           curd  0.000083
1  specialty bar  0.000083
2     rolls/buns  0.000057
3        dessert  0.000043
4  domestic eggs  0.000043
```

Case 2: Recommendation example without Frequent Pattern Mining and using only Collaborative Filtering

In [23]:
```
# Valid user and without FPM.
recommendation_sys.run()
```

```
Top 5 recommendations for user 1000 without Frequent Pattern Mining:
================================================================================
           Items     Score
0           curd  0.000083
1  specialty bar  0.000083
2     rolls/buns  0.000057
3        dessert  0.000043
4  domestic eggs  0.000043
```

Case 3: Recommendation example when the user has no purchase history

In [24]:
```
# User without purchase history.
recommendation_sys.run()
```

```
User 1008 does not have a purchase history.
```

Case 4: Recommendation example with invalid user ID

In [25]:
```
# Invalid User ID.
recommendation_sys.run()
```

```
User 50000 does not exist in the database. Please try another user id.
```

Case 5: Recommendation examples using only Frequent Pattern Mining

In [26]:
```
# Recommendations using only FPM
recommendation_sys.get_pattern_recommendation_examples()
```

```
Top 5 patterns
======================================================================
```

	antecedents	consequents	support_train	confidence_train	support_test	confidence_test
0	(domestic eggs, meat spreads)	(whole milk)	0.002004	1.0	0.000929	1.000000
1	(white bread, brown bread, bottled water)	(whole milk)	0.002004	1.0	0.000619	0.500000
9	(rolls/buns, white bread, yogurt, soda)	(whole milk)	0.001718	1.0	0.000929	0.600000
3	(white bread, ham)	(whole milk)	0.001718	1.0	0.001857	0.461538
7	(other vegetables, sugar, pip fruit)	(whole milk)	0.001718	1.0	0.000929	0.333333

```
Example recommendations using the top 5 patterns
======================================================================
```

	pattern	User_id	recommended_items	score
0	{'domestic eggs', 'meat spreads'} => {'whole milk'}	2394	[]	1.0
1	{'domestic eggs', 'meat spreads'} => {'whole milk'}	2990	[whole milk]	1.0
2	{'white bread', 'brown bread', 'bottled water'} => {'whole milk'}	1997	[whole milk]	1.0
3	{'white bread', 'brown bread', 'bottled water'} => {'whole milk'}	2433	[whole milk]	1.0
4	{'rolls/buns', 'white bread', 'yogurt', 'soda'} => {'whole milk'}	2613	[whole milk]	1.0
5	{'rolls/buns', 'white bread', 'yogurt', 'soda'} => {'whole milk'}	3050	[whole milk]	1.0
6	{'white bread', 'ham'} => {'whole milk'}	1237	[whole milk]	1.0
7	{'white bread', 'ham'} => {'whole milk'}	1360	[whole milk]	1.0
8	{'other vegetables', 'sugar', 'pip fruit'} => {'whole milk'}	2055	[whole milk]	1.0
9	{'other vegetables', 'sugar', 'pip fruit'} => {'whole milk'}	2517	[whole milk]	1.0

8. References

1. Dickey, D. & Fuller, W. 1979, 'Distribution of the Estimators for Autoregressive Time Series With a Unit Root', *Journal of the American Statistical Association*, vol. 74, DOI:10.2307/2286348.
2. Jadon, A. & Patil, A. 2025, 'A Comprehensive Survey of Evaluation Techniques for Recommendation Systems', in Bairwa, A.K., Tiwari, V., Vishwakarma, S.K., Tuba, M. & Ganokratanaa, T. (eds), *Computation of Artificial Intelligence and Machine Learning*, Springer Nature, Cham, pp. 281–304, DOI:10.1007/978-3-031-71484-9_25.
3. Krzywicki, A. 2025, 'Recommendation Systems', Lecture slides distributed for the week 10 of COMP SCI 7306 Mining Big Data course, The University of Adelaide on 07 April 2025.
4. Lee, C.H., Kim, Y.H. & Rhee, P.K. 2001, 'Web personalization expert with combining collaborative filtering and association rule mining technique', *Expert Systems with Applications*, vol. 21, no. 3, pp. 131–137, DOI:10.1016/S0957-4174(01)00034-3.
5. Parvatikar, S. & Joshi, B. 2015, 'Online book recommendation system by using collaborative filtering and association mining', in *2015 IEEE International Conference on Computational Intelligence and Computing Research (ICCIC)*, pp. 1–4, DOI:10.1109/ICCIC.2015.7435717.
6. Saraei, T., Benali, M. & Frayret, J.M. 2025, 'A hybrid recommendation system using association rule mining, i-ALS algorithm, and SVD++ approach: A case study of a B2B company', *Intelligent Systems with Applications*, vol. 25, DOI:10.1016/j.iswa.2025.200477.
7. Wikipedia contributors 2025, *Pareto principle*, Wikipedia, viewed 19 April 2025, <https://en.wikipedia.org/wiki/Pareto_principle/ >.

9. Appendix

9.1 Libraries used

- matplotlib v3.7.5
- mlxtend v0.23.4
- numpy v1.26.4
- pandas v2.2.3
- scikit-learn v3.7.5

- seaborn v0.12.2
- statsmodels v0.14.4
- tqdm v4.66.4